

What a Determinant’s Derivative Knows

Jacobi’s formula and Newton’s identity for matrix traces in Lean 4, with an application to the Ihara counts

Carles Marín*

Independent researcher

karlesmarin@gmail.com

June 2026

Abstract

A one-line rule from a linear-algebra course—the *derivative of a determinant is the trace of the adjugate times the derivative of the matrix*—turns out to be the seed from which the characteristic polynomial’s coefficients, the power sums of a matrix’s traces, and ultimately the closed-walk counts of a graph all grow. This paper formalizes that seed and its first fruits in Lean 4 / Mathlib, all **sorry-free**. The headline is *Jacobi’s formula* $(\det M)' = \text{tr}(\text{adj } M \cdot M')$ for a matrix of polynomials, for which I could find no prior proof in any proof assistant; Mathlib had only the analytic derivative of $\det$ between normed spaces, never the algebraic identity. From it I derive *Newton’s identity for matrix traces*—the logarithmic derivative of the reversed characteristic polynomial collects the trace power sums—an all-orders statement that Mathlib provided only at the linear coefficient, and that needs no eigenvalues. The engine is a *resolvent series* $(I - XM)^{-1} = \sum_k M^k X^k$ over formal power series, also new to the library. As an application I compose these with my earlier formalization of Bass’s determinant formula to read the Ihara non-backtracking counts $N_k = \text{tr}(B^k)$ off the adjacency spectrum, and to prove combinatorially that $\text{tr}(B^k)$ counts closed non-backtracking walks. I am exact about the boundary: none of this is new mathematics; the prime-geodesic counting that the cyclic structure sets up is built only as far as the rotation-invariant weight, not the Euler product; and $\text{tr}(B^k)$ counts *rooted* closed walks, not geodesic classes. The contribution is the formalization, and a small piece of certified infrastructure that any future development of characteristic polynomials, zeta functions, or expander spectra can stand on.

Reader’s map. If you read only two things, read Theorem 1 (Jacobi) and Theorem 2 (Newton): the first says a determinant’s derivative is a trace; the second says, taken to all orders, that single fact *is* the bridge between a matrix’s traces and its characteristic polynomial. Everything else—the resolvent engine (Section 3), the walk count (Section 5), the Ihara application (Section 6)—hangs from those two. The load-bearing formulas are framed; the honest limits are stated as plainly as the results.

1 Introduction: a short history, and why this paper

Could a rule for differentiating a determinant—a fact so elementary it is often set as an exercise—be the engine that, taken to all orders, recovers the characteristic polynomial from a matrix’s traces,

*Formalized with AI assistance (Claude, Anthropic); the mathematics and all claims are the author’s responsibility. The methodology is discussed in Section 8.

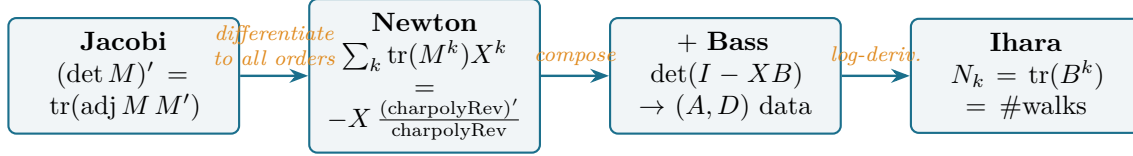


Figure 1: The spine of the paper. One elementary fact—a determinant’s derivative is a trace (Jacobi)—taken to all orders becomes the dictionary between a matrix’s traces and its characteristic polynomial (Newton), and composed with Bass’s determinant formula reaches the closed non-backtracking walk counts $N_k = \text{tr}(B^k)$ of a graph. Each arrow is a theorem of this paper.

and then counts the closed walks of a graph? The answer is yes, and following it requires three short histories to meet.

The first is *Jacobi’s formula*. In the nineteenth century Jacobi recorded how the determinant of a matrix changes as its entries vary: $d(\det M) = \text{tr}(\text{adj } M \cdot dM)$, where $\text{adj } M$ is the adjugate (the transpose of the cofactor matrix). It is the matrix calculus behind Liouville’s formula for the Wronskian of a linear ODE, behind the Faddeev–LeVerrier algorithm that computes a characteristic polynomial from traces, and behind every first-order perturbation of an eigenvalue. It is also, in its *algebraic* form—over an arbitrary commutative ring, with the formal derivative of polynomials rather than a limit—absent from the largest formal library of mathematics. Mathlib knew how to differentiate a determinant *analytically*, as a smooth map between normed spaces; it did not know the one-line algebraic identity.

The second is *Newton’s identities*. Newton related the power sums $p_k = \sum_i \lambda_i^k$ of a collection of numbers to their elementary symmetric functions e_k —equivalently, the coefficients of the polynomial $\prod_i (X - \lambda_i)$. Specialized to the eigenvalues of a matrix, the power sums are exactly the traces $\text{tr}(M^k)$, and the elementary symmetric functions are exactly the coefficients of the characteristic polynomial. So Newton’s identities, in this guise, are a bridge: they let you compute the characteristic polynomial from the traces of powers, and conversely. Mathlib has Newton’s identities, but only in the abstract—for symmetric polynomials in free variables—never connected to a matrix’s traces. The bridge existed; its matrix end was missing, recorded only at the very first coefficient as $\text{tr } M = -(\text{next coefficient of the charpoly})$.

The third thread ties this to graphs, and to my earlier work. The Ihara zeta function of a graph counts its closed non-backtracking walks; its reciprocal is the determinant $\det(I - XB)$ of Hashimoto’s non-backtracking operator B , and I formalized *Bass’s* reduction of that determinant to vertex-sized data in a companion paper [7]. The counts $N_k = \text{tr}(B^k)$ are exactly the trace power sums that Newton’s identity governs. The same derivative that powers Jacobi and Newton therefore reaches, when applied to Bass’s identity, all the way to the cycle counts of a graph.

This paper formalizes that chain—Jacobi, then Newton, then the Ihara counts—in Lean 4, and is exact about where the road, for now, ends.

2 Jacobi’s formula

If you nudge one entry of a matrix, how does its determinant move—and why should the answer involve the trace of anything?

For a matrix M whose entries are polynomials in a formal variable, write M' for the entrywise derivative and $\det M$ for the determinant (a polynomial). The answer is a trace.

Theorem 1 (Jacobi’s formula, in Lean 4). *For an $n \times n$ matrix M of polynomials over a commutative ring,*

$$(\det M)' = \text{tr}(\text{adj } M \cdot M').$$

The statement is sorry-free (`Matrix.derivative_det`); `#print axioms` reports only `propext`, `Classical.choice`, `Quot.sound`.

The proof is the determinant’s own multilinearity, made one column at a time. Differentiating $\det M = \sum_{\sigma} \varepsilon_{\sigma} \prod_i M_{\sigma(i),i}$ by the Leibniz rule and regrouping shows that $(\det M)'$ is the sum, over columns k , of the determinant of M with column k replaced by its derivative; Cramer’s rule reads each of those single-column determinants off the adjugate, and the sum reassembles as a trace. (In Lean the regrouping uses the product form of the Leibniz formula, `det_apply'`, to keep every coefficient a genuine ring element rather than a signed permutation—a small choice that makes the whole proof a chain of rewrites.)

Why this is useful, plainly. A determinant’s derivative appears wherever a matrix moves. Three places it earns its keep: *Liouville’s formula*—the Wronskian of a linear system evolves as $\det(\Phi)' = \text{tr}(A) \det(\Phi)$, which is Jacobi’s formula for $\Phi' = A\Phi$; *eigenvalue perturbation*—the first-order change of a simple eigenvalue is a trace against its spectral projection; and the *Faddeev–LeVerrier algorithm*, which extracts a characteristic polynomial and a matrix inverse from nothing but traces of powers, and whose correctness rests on exactly this identity. Each is a theorem someone may one day wish to certify; each now has its first stone.

3 The resolvent series: an eigenvalue-free engine

Can one talk about $(I - XM)^{-1}$ without ever inverting anything—and without ever mentioning an eigenvalue?

Over formal power series, yes. The geometric series $\sum_k M^k X^k$ is an honest matrix of power series, and it is the inverse of $I - XM$.

Definition 1 (Resolvent series). *For an $n \times n$ matrix M over a commutative ring, the resolvent series is the matrix of formal power series $(\sum_k M^k X^k)$, written `resolventSeries M`.*

Lemma 1 (The resolvent inverts $I - XM$, and its trace is the trace generating function).

$$(I - XM) \cdot \sum_k M^k X^k = I, \quad \text{tr}(\sum_k M^k X^k) = \sum_k \text{tr}(M^k) X^k.$$

Both are sorry-free (`one_sub_smul_mul_resolventSeries`, `trace_resolventSeries`). The inverse identity is proved from the fixed-point equation $R = I + XM \cdot R$ by a single rearrangement—no eigenvalues, no algebraic closure, over any commutative ring.

This is the engine of Section 4. Its point is exactly its modesty: the standard way to relate traces of powers to the characteristic polynomial passes through eigenvalues, and so needs the matrix to split—an algebraically closed field, or a generic-matrix detour. The resolvent series dispenses with all of it. The trace of $(I - XM)^{-1}$ is, by construction, the *trace generating function* $\sum_k \text{tr}(M^k) X^k$, and that is the object Newton’s identity is about.

Why this is useful, plainly. A formal resolvent is the natural home of any quantity one wants to read “order by order” in a perturbation: the moments $\text{tr}(M^k)$ of a matrix, the Green’s function of a discrete operator, the generating function of walks (Section 5). Having $(I - XM)^{-1}$ available as certified formal-power-series infrastructure—rather than re-deriving the geometric series each time—is a small convenience that compounds.

4 Newton’s identity for matrix traces

The traces $\text{tr}(M), \text{tr}(M^2), \dots$ and the coefficients of the characteristic polynomial are two descriptions of the same matrix. What, exactly, is the dictionary between them?

It is a single power-series identity, and it is Jacobi’s formula taken to all orders. Write `charpolyRev M` for the reversed characteristic polynomial $\det(I - XM)$ —a polynomial with constant term 1, hence a unit in the power-series ring.

Theorem 2 (Newton’s identity for matrix traces, in Lean 4). *For an $n \times n$ matrix M over a commutative ring, in the ring of formal power series,*

$$(\text{charpolyRev } M)' = - \text{charpolyRev } M \cdot \sum_{k \geq 0} \text{tr}(M^{k+1}) X^k.$$

*Equivalently $\sum_{k \geq 1} \text{tr}(M^k) X^k = -X (\text{charpolyRev } M)' / \text{charpolyRev } M$, the logarithmic derivative. The statement is **sorry-free** (`Matrix.charpolyRev_logDeriv`), over any commutative ring, eigenvalue-free. At the linear coefficient it specializes to the lemma Mathlib already had, $\text{tr } M = -[X^1] \text{charpolyRev } M$.*

The proof is the chain promised in the introduction. Differentiate $\text{charpolyRev } M = \det(I - XM)$ by Jacobi (Theorem 1); the derivative of $I - XM$ is $-M$, so $(\text{charpolyRev } M)'$ is $-\text{tr}(\text{adj}(I - XM) \cdot M)$. The adjugate of $I - XM$ is $\det(I - XM)$ times its inverse—which, by Lemma 1, is $\text{charpolyRev } M$ times the resolvent series; the trace of the resolvent against M is the shifted generating function $\sum_k \text{tr}(M^{k+1}) X^k$. Transporting the polynomial Jacobi identity along the coercion into power series and substituting closes the loop. The whole argument never names an eigenvalue.

Why this is useful, plainly. This is the dictionary itself. Reading it one way, it computes the coefficients of the characteristic polynomial—and hence the determinant, the trace, and (via Cayley–Hamilton) the inverse—from the traces of powers; reading it the other way, it computes those traces from the polynomial. It is the matrix-trace form of Newton’s identities, the form a working mathematician actually meets, and the form that Mathlib’s symmetric-function version did not reach. The logarithmic-derivative shape $\sum \text{tr}(M^k) X^k = -X(\text{charpolyRev})' / \text{charpolyRev}$ is, moreover, the universal pattern behind *zeta functions*: a determinant’s reciprocal, log-differentiated, yields a count—which is precisely how Section 6 reaches the graph.

5 Matrix powers count walks

Before counting cycles, what does a single entry of M^k count?

The classical answer—walks—needs a formal home, and for *directed* graphs Mathlib did not have one. It records that a power of a *simple* (undirected) graph’s adjacency matrix counts walks; the directed analogue, which the non-backtracking operator demands, was missing.

Lemma 2 (Powers count walks). *For a matrix M over a commutative semiring, the (i, j) entry of M^k is the sum, over length- k vertex walks from i to j , of the product of the edge weights (`pow_apply_eq_sum`). For the 0–1 adjacency matrix of a decidable relation—a finite directed graph—this is a genuine count: $((\text{adjacency})^k)_{ij}$ is the number of length- k directed walks from i to j (`relMatrix_pow_apply`). Both are *sorry-free*.*

The trace then counts *closed* walks, and—summing the diagonal—closed walks rooted at a vertex. The same statement, reorganized over the cyclic index $\mathbb{Z}/(m+1)$, becomes a sum over *cyclic* walks (`trace_pow_eq_sum_cyclicProd`), and the weight of a cyclic walk is invariant under rotating its basepoint (`cyclicProd_comp_finRotate`, and under the whole rotation group). That rotation invariance is the door to geodesics—closed walks counted up to cyclic rotation—though, as Section 7 is careful to say, only the door.

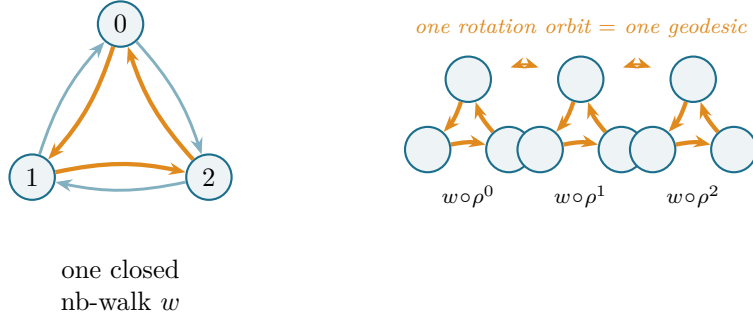


Figure 2: Closed non-backtracking walks, and the rotation that groups them. *Left:* the triangle K_3 has six darts; one closed non-backtracking walk of length 3 is shown in amber (the reverse darts, in grey, would backtrack). *Right:* rotating the basepoint by the cyclic shift $\rho = \text{finRotate}$ permutes the three rooted walks of this orbit while leaving the cyclic weight fixed (`cyclicProd_comp_finRotate`)—so a single *geodesic* accounts for an orbit of rooted walks. Here $\text{tr}(B^3) = 6$: two orbits (the cycle and its reverse), three basepoints each.

Why this is useful, plainly. “Entry of a matrix power = number of walks” is the workhorse of spectral graph theory: mixing times, the method of moments for eigenvalue distributions, expander estimates, and the trace formulas this paper composes all rest on it. Having it for *directed* graphs, not only undirected ones, is what lets the non-backtracking operator—an inherently directed object on oriented edges—be counted at all.

6 The Ihara counts, composed

If Jacobi differentiates a determinant into traces, and Bass collapses the Ihara determinant to vertex data, what does their composition say?

It says the non-backtracking counts of a graph are spectral. Let B be Hashimoto’s operator, A the adjacency and D the degree matrix; Bass’s formula [7] reads $(1 - X^2)^{|V|} \det(I - XB) = (1 - X^2)^{|E|} \det(I - XA + X^2(D - I))$. Differentiating both sides (Theorem 1, on each determinant) and substituting Newton’s identity (Theorem 2) on the non-backtracking side isolates the count generating function.

Theorem 3 (The Ihara N_k identity, in Lean 4). *With $Q = \det(I - XA + X^2(D - I))$, in the power-series ring,*

$$\sum_k \text{tr}(B^{k+1})X^k = \frac{\det(I - XB) ((1 - X^2)^{|V|})' - (1 - X^2)^{|E|}Q' - Q ((1 - X^2)^{|E|})'}{(1 - X^2)^{|V|} \det(I - XB)}.$$

The two $(1 - X^2)$ -powers and both determinants are units, so the division is the formal `invOfUnit`; the identity is `sorry-free` (`ihara_Nk_explicit`). It expresses the non-backtracking counts $N_k = \text{tr}(B^k)$ purely through the adjacency data (A, D) .

And the counts are combinatorial. Because B is the 0–1 adjacency matrix of the non-backtracking relation on darts ($d \rightsquigarrow e$ when the head of d is the tail of e and e is not the reverse of d), Lemma 2 specializes:

Theorem 4 ($\text{tr}(B^k)$ counts non-backtracking walks, in Lean 4).

$$\text{tr}(B^k) = \#\{\text{closed non-backtracking walks of length } k, \text{ rooted at a dart}\}.$$

`sorry-free` (`trace_hashimoto_pow`).

So the same derivative that began as a one-line linear-algebra fact ends, four compositions later, as the statement that a graph’s non-backtracking walks of length k number $\text{tr}(B^k)$, and that this number is computable from the graph’s adjacency spectrum alone.

Why this is useful, plainly. Non-backtracking counts and spectra underpin sharp epidemic and percolation thresholds, spectral community detection, and—through the Ihara analogue of the Riemann hypothesis—the very definition of a Ramanujan graph as an optimal expander. Theorem 3 certifies the bridge between those counts and the adjacency data they are usually computed from; Theorem 4 certifies that the counts mean what they are said to mean.

Table 1: Every machine-checked ingredient. All are `sorry-free`; `#print axioms` reports only `[propext, Classical.choice, Quot.sound]`. Sources: `Ihara/TraceFormula.lean` and `MathlibPR/MatrixWalkCount.lean`.

Lean name	Statement	§
<code>Matrix.derivative_det</code>	$(\det M)' = \text{tr}(\text{adj } M \cdot M')$ (Jacobi)	2
<code>one_sub_smul_mul_resolventSeries</code>	$(I - XM) \sum_k M^k X^k = I$	3
<code>trace_resolventSeries</code>	$\text{tr} \sum_k M^k X^k = \sum_k \text{tr}(M^k) X^k$	3
<code>Matrix.charpolyRev_logDeriv</code>	$(\text{charpolyRev } M)' = -\text{charpolyRev } M \sum_k \text{tr}(M^{k+1}) X^k$ (Newton)	4
<code>pow_apply_eq_sum</code>	$(M^k)_{ij} = \sum_{\text{walks}} \prod(\text{weights})$	5
<code>relMatrix_pow_apply</code>	directed walk count, 0–1 case	5
<code>trace_pow_eq_sum_cyclicProd</code>	$\text{tr}(M^{m+1}) = \sum_{\text{cyclic } w} \text{cyclicProd}$	5
<code>cyclicProd_comp_finRotate_iterate</code>	rotation invariance (full group)	5
<code>ihara_Nk_explicit</code>	$\sum_k \text{tr}(B^{k+1}) X^k$ from (A, D)	6
<code>trace_hashimoto_pow</code>	$\text{tr}(B^k) = \#$ closed nb-walks	6

7 What I claim, and what I do not

The mathematics is classical throughout: Jacobi’s formula, Newton’s identities, the walk count, and Bass’s theorem are all old. The contribution is the formalization, and I separate the two on purpose.

Table 2: Independent numerical cross-checks (SymPy), exact symbolic equality. The graph row is the triangle K_3 : the trace of B^k matches a brute-force enumeration of rooted closed non-backtracking walks, 0, 0, 6, 0, 0, 6 for $k = 1, \dots, 6$ (the cycle and its reverse, three basepoints each).

Identity	Instance	Result
Jacobi $(\det M)' = \text{tr}(\text{adj } M M')$	3×3 matrix of polynomials	exact
Newton (log-derivative)	3×3 and 2×2 , mod X^8	exact
$\text{tr}(B^k) = \#\{\text{rooted nb-walks}\}$	$K_3, k = 1, \dots, 6$	0, 0, 6, 0, 0, 6 both
Bass $(1-u^2)^{ V } \det(I-uB) = \dots$	K_3	exact

What I claim as new (as formalization). (N1) the first machine-checked proof I am aware of, in any proof assistant, of Jacobi’s *algebraic* formula $(\det M)' = \text{tr}(\text{adj } M \cdot M')$; (N2) the *matrix-trace* form of Newton’s identities, $(\text{charpolyRev } M)' = -\text{charpolyRev } M \sum_k \text{tr}(M^{k+1})X^k$, which Mathlib had only at the linear coefficient; (N3) the resolvent series $(I - XM)^{-1} = \sum_k M^k X^k$ and the *directed*-graph walk count, neither in the library. All are formalization contributions; none is new mathematics.

And, plainly, what this is not. $\text{tr}(B^k)$ counts *rooted* closed non-backtracking walks—walks with a chosen basepoint dart—*not* closed geodesics, which are the cyclic equivalence classes of such walks, and not the prime geodesics whose Euler product is the analytic Ihara zeta. The cyclic structure that distinguishes them *is* set up here: the trace is rewritten as a sum over cyclic walks, and the weight is proved invariant under the whole rotation group, so each weight is a class function on the rotation orbits. But the orbit-counting, the primitive-period bookkeeping, the Möbius inversion, and the Euler product that turn rooted counts into prime-geodesic counts are *not* built. I have laid the combinatorial floor; the number-theoretic superstructure is mapped, not raised.

8 Implications

Three kinds, in decreasing order of how firmly I can claim them, and a methodological aside.

Reusable, certified building blocks. Jacobi’s algebraic formula, the resolvent series, the matrix-trace Newton identity, and the directed walk count are now machine-checked and available to anyone formalizing characteristic polynomials, the Faddeev–LeVerrier algorithm, zeta functions of graphs, or expander spectra. Mathlib had none of the four in usable form. This is the implication I can state most firmly, and the one I would point a future formalizer to first.

A foothold for what these unlock. Newton’s identity is the gateway to computing characteristic-polynomial coefficients from traces and back; Jacobi underlies Faddeev–LeVerrier and Liouville’s formula; the Ihara composition is one step from the prime-geodesic counting it sets up. Each is a landmark these stones could support.

The mathematics, with a note of caution. These identities matter because the objects downstream of them—characteristic polynomials, non-backtracking spectra, Ramanujan expanders—matter. I am wary of borrowing that importance: certifying a foundational identity ships no algorithm and no network. Its value is in certified foundations and in the growing corpus of formalized mathematics, not in immediate application; I claim the former, not the latter.

A methodological aside. This development was carried out with an AI system as a research instrument: it proposed definitions, lemmas and tactics, and a build-as-oracle loop verified or rejected each against the kernel. The careful accounting here—every claim sized to exactly what was proved, the rooted-versus-geodesic distinction drawn as sharply as the theorems—is part of what I hope the work shows: that AI-assisted formalization can resist the over-claiming the method invites, precisely because the proof assistant, not the assistant, has the last word.

9 Where this stops being useful

Honesty cuts both ways, so the limits are stated as plainly as the results. Theorem 3 is a *generating-function* identity, not the analytic zeta; it locates no pole and names no prime. Theorem 4 counts *rooted* walks, and the gap to geodesics—an entire necklace-counting and Möbius-inversion theory—is real, not cosmetic. Newton’s identity is proved over a commutative ring but says nothing about when the characteristic polynomial *splits*; Jacobi is for matrices of polynomials, and the smooth-manifold version remains a separate object. This paper is a handful of certified stones near the start of a long road, and I would rather name the road than overstate the stones.

10 Related work

The mathematics is classical: Jacobi’s formula and Liouville’s, Newton’s identities and the Faddeev–LeVerrier algorithm built on them, the walk-counting reading of matrix powers, and—for the application—Ihara’s zeta function [3], Hashimoto’s edge operator [2], Bass’s determinant formula [1], and Terras’s modern account [8]. On the formalization side, Mathlib [4] provides determinants, the adjugate, Cramer’s rule, the reversed characteristic polynomial (at the linear coefficient only), Newton’s identities for symmetric polynomials, and walk counts for simple graphs—but not Jacobi’s algebraic formula, the matrix-trace Newton identity, the resolvent series, or the directed walk count. This development is a companion to my earlier Lean formalizations of the matching polynomial [5, 6] and of Bass’s formula [7]; I am not aware of a prior machine-checked proof of Jacobi’s algebraic formula or of the matrix-trace Newton identity in any proof assistant.

11 Conclusion

I set out from the most elementary fact I could name—how a determinant moves when its matrix moves—and followed it, in a proof assistant, through Newton’s dictionary between traces and characteristic polynomials, and on to the closed-walk counts of a graph. None of this is new mathematics; the contribution is that Jacobi’s algebraic formula, the matrix-trace Newton identity, the resolvent series, and the directed walk count are now machine-verified where I could find no prior formalization, and that they compose, all the way to the Ihara counts. The prime-geodesic counting they set up, and full spectral generality, remain ahead—mapped, but unraised. I offer this as a pearl and an invitation.

Artifacts. All Lean sources (sorry-free, depending only on the three standard axioms), are available at

<https://github.com/karlesmarin/godsil-gutman-lean>

in `Ihara/TraceFormula.lean` (Jacobi, Newton, the Ihara composition) and `MathlibPR/MatrixWalkCount.lean` (the resolvent series, the walk count, the cyclic structure). Verify the axiom footprint

with, e.g., `#print axioms Matrix.charpolyRev_logDeriv ~> [propext, Classical.choice, Quot.sound]`.

References

- [1] H. Bass, *The Ihara–Selberg zeta function of a tree lattice*, Internat. J. Math. **3** (1992), 717–797.
- [2] K. Hashimoto, *Zeta functions of finite graphs and representations of p -adic groups*, in *Automorphic forms and geometry of arithmetic varieties*, Adv. Stud. Pure Math. **15** (1989), 211–280.
- [3] Y. Ihara, *On discrete subgroups of the two by two projective linear group over p -adic fields*, J. Math. Soc. Japan **18** (1966), 219–235.
- [4] The mathlib Community, *The Lean mathematical library*, in *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP 2020)*, 367–381.
- [5] C. Marín, *Random Signs into Matchings: a machine-checked Godsil–Gutman identity in Lean 4* (2026).
- [6] C. Marín, *Unfolding a Graph into a Tree: Heilmann–Lieb real-rootedness in Lean 4* (2026).
- [7] C. Marín, *Folding Edges into Vertices: a machine-checked proof of Bass’s determinant formula for the Ihara zeta in Lean 4* (2026), Zenodo DOI 10.5281/zenodo.20573120.
- [8] A. Terras, *Zeta Functions of Graphs: A Stroll through the Garden*, Cambridge Studies in Advanced Mathematics **128**, Cambridge Univ. Press, 2011.